Naive RAG Experiments

Overview

The naive RAG configuration represents the most straightforward implementation of retrieval-augmented generation: a user question is embedded and used to retrieve the top-k most similar document chunks from the vector store, which are then concatenated into a context block and passed to the language model for answer generation. Three experiments were conducted under this configuration, varying the number of retrieved chunks (k = 3, k = 5, and k = 8) to understand how retrieval breadth affects generation quality.

Pipeline

The end-to-end pipeline follows four steps:

1. A question from the evaluation set is passed to the embedding model, producing a dense query vector.
2. The query vector is compared against all stored chunk embeddings in ChromaDB using cosine similarity. The top-k most similar chunks are retrieved.
3. The retrieved chunks are concatenated and inserted into a RAG prompt template alongside the original question.
4. The completed prompt is sent to the language model, which generates a response grounded in the retrieved context.

The prompt template used for generation was:

> Use the following research contexts to answer the question. Answer based only on the provided context. Be precise and evidence-based.

The embedding model used for both document ingestion and query encoding is `sentence-transformers/all-MiniLM-L6-v2` . The language model is Llama 3.3 70B Versatile, accessed via the Groq inference API. ChromaDB was chosen as the primary vector store for these experiments, with cosine similarity as the retrieval metric.

Parallelisation Strategy

Evaluating 200 questions through an external inference API within practical time constraints required a parallelised execution strategy. The approach partitions the evaluation set into contiguous row slices, assigning each slice to a dedicated Groq API key and processing all slices simultaneously using Python's `ThreadPoolExecutor`.

Each thread initialises its own `ChatGroq` client and its own RAG chain, so no shared mutable state exists between threads. The vector store retriever is read-only and safe to call from multiple threads concurrently. Row ordering is preserved by indexing each result slice back to its original position before concatenating.

`asyncio` was deliberately avoided for this task. Jupyter environments run a persistent event loop, which means any call to `asyncio.run()` from within a notebook cell raises a `RuntimeError`. Since the Groq client exposes only a synchronous `invoke()` method, wrapping calls in coroutines would not provide additional concurrency. Network I/O releases the Python GIL, so `ThreadPoolExecutor` achieves genuine parallelism across threads without requiring an asynchronous interface.

In cases where individual rows failed due to transient rate limit errors during a parallel run, those rows were identified, re-run as a small separate batch, and merged back into the main result frame before evaluation.

Evaluation Metrics

Each experiment was evaluated using a combination of RAGAS and DeepEval metrics.

The RAGAS metrics used are all non-LLM-based, meaning they do not require an additional API call to an LLM judge and instead rely on lexical and structural comparison:

- **NonLLMContextRecall** measures what fraction of the information in the golden reference contexts appears in the retrieved chunks. It is a coverage metric: a high score means the retriever brought back passages that contain the relevant information.
- **NonLLMContextPrecisionWithReference** measures what fraction of the retrieved chunks were actually relevant to the question given the reference. A high score means the retriever was selective and did not return a large proportion of irrelevant material.
- **BLEU Score** measures n-gram precision between the generated answer and the golden answer.
- **ROUGE-L (F-measure)** measures the longest common subsequence between the generated and golden answers, normalised by a harmonic mean of precision and recall.

The DeepEval metrics are LLM-judged and provide semantically richer assessments:

- **ContextualRecall** measures whether the generated answer is grounded in the retrieved context.
- **ContextualPrecision** measures whether the retrieved context is relevant to the question.

- **Faithfulness** measures whether the generated answer contains claims that can be verified against the retrieved passages without hallucination.
- **AnswerRelevancy** measures whether the generated answer is topically responsive to the question.

DeepEval metrics were computed using an LLM judge (`openai/gpt-oss-120b` via Groq) and were parallelised using the same multi-key strategy as the RAG inference itself.

Experiment 1: k = 3

In the first experiment, the retriever was configured to return the top 3 chunks for each query. This is the most conservative retrieval setting, keeping the context window concise and reducing the risk of injecting off-topic passages.

The 200-question evaluation set was split across 10 API keys at 20 rows per key. All 200 answers were collected successfully in a single parallel run.

Results

Metric	Score
NonLLMContextRecall	0.1804
NonLLMContextPrecisionWithReference	0.2979
BLEU Score	0.1853
ROUGE-L (F-measure)	0.3098

At k = 3, the context recall is relatively low at 0.18, indicating that only a modest fraction of the information in the reference passages was recovered by the retriever. The context precision at 0.30 suggests that roughly a third of retrieved chunks were judged relevant to the question given the reference. The BLEU and ROUGE-L scores show a clear improvement over the vanilla LLM baseline (BLEU 0.0757, ROUGE-L 0.2113), confirming that even minimal retrieval grounding improves lexical alignment with the reference answers.

Experiment 2: k = 5

The second experiment increased the number of retrieved chunks to 5. This provides the model with more context and gives the retriever a wider net to capture relevant passages, at the cost of potentially including more irrelevant material in the prompt.

For this run, 16 API keys were used, with 13 rows assigned per key. Two rows failed during the initial parallel run due to transient token-per-minute rate limit errors on individual keys. These rows were re-run in a separate small batch and merged back into the main result frame before evaluation, resulting in a complete 200-sample evaluation set.

Results

Metric	Score
NonLLMContextRecall	0.2008
NonLLMContextPrecisionWithReference	0.3067
BLEU Score	0.1737
ROUGE-L (F-measure)	0.2966

Increasing k to 5 improved context recall from 0.1804 to 0.2008, reflecting the expected behaviour: a wider retrieval net captures a larger share of the reference information. Context precision also increased slightly to 0.3067. However, the generation quality metrics show a modest decrease compared to k = 3: BLEU dropped from 0.1853 to 0.1737 and ROUGE-L dropped from 0.3098 to 0.2966. This suggests that while more context improves coverage, the additional passages may introduce some noise that modestly dilutes the precision of the generated answer relative to the reference wording.

Experiment 3: k = 8

The third experiment retrieved the top 8 chunks, the most expansive retrieval window tested under the naive RAG configuration. At this setting, the context block injected into the prompt is substantially longer, covering a wider range of passages but also carrying greater risk of off-topic content.

The same 16-key parallelisation setup was used. One row failed during the initial run and was recovered via a targeted re-run before evaluation.

Results

Metric	Score
NonLLMContextRecall	0.2137
NonLLMContextPrecisionWithReference	0.3061
BLEU Score	0.1797
ROUGE-L (F-measure)	0.3051

At k = 8, context recall reaches its highest value across the three naive RAG experiments (0.2137), confirming that broader retrieval continues to improve information coverage. Context precision remains approximately stable at 0.3061, slightly below the k = 5 figure, suggesting that the additional chunks retrieved at k = 8 are marginally less relevant on average. The generation metrics (BLEU 0.1797, ROUGE-L 0.3051) partially recover compared to k = 5 and are broadly comparable to k = 3, indicating that the model can make reasonable use of the larger context despite its added length.

Summary

The three naive RAG experiments demonstrate a consistent pattern: increasing k improves retrieval coverage (context recall) monotonically, while context precision and generation quality metrics are less sensitive to k and show modest variation across configurations. Across all three settings, the naive RAG pipeline substantially outperforms the vanilla LLM baseline on BLEU and ROUGE-L, confirming that document retrieval provides meaningful grounding for the generation step even under the simplest possible retrieval strategy.

Configuration	Context Recall	Context Precision	BLEU	ROUGE-L
k = 3	0.1804	0.2979	0.1853	0.3098
k = 5	0.2008	0.3067	0.1737	0.2966
k = 8	0.2137	0.3061	0.1797	0.3051
Vanilla LLM	—	—	0.0757	0.2113

The overall scores remain moderate, which is expected given the difficulty of biomedical QA and the relatively simple retrieval mechanism. These results serve as the baseline for the more advanced retrieval strategies explored in subsequent experiments.